

BinX TECH • AI & MACHINE LEARNING INTERNSHIP PROGRAM

WEEK 3

SUPERVISED LEARNING

Regression & Classification with Scikit-learn

The first week of Phase 2: training the first real machine learning models. Interns build regression models to predict numbers and classification models to predict categories — linear and logistic regression, decision trees, random forests, SVMs, and k-NN — all using the Scikit-learn API.

PHASE 2 • 40 HOURS • 5 TRAINING DAYS • HANDS-ON

Prepared by BinX Tech
Palestine | Nablus

BinX
TECH

1. Week 3 Overview

Week	Week 3 of 10 — Phase 2: Core ML Training
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 5-6 combined)
Format	On-site / Remote / Hybrid — all work in Jupyter notebooks committed to GitHub
Focus	Supervised learning: linear & logistic regression, decision trees, random forests, SVMs, k-NN — using the Scikit-learn API
Prerequisite	Weeks 1-2 completed: Python/NumPy/Pandas fluency, math foundations, EDA methodology
Mentor Supervision	Daily check-in; the Day 5 mini-project (a trained, evaluated model) is the first graded model deliverable

2. Week 3 Learning Objectives

- Explain what supervised learning is and distinguish regression from classification.
- Split data correctly into training and test sets and explain why this prevents misleading results.
- Train, predict with, and evaluate a linear regression model and interpret its coefficients and error.
- Train and evaluate a logistic regression classifier and read a confusion matrix.
- Train and compare tree-based models (decision trees, random forests) plus SVM and k-NN.
- Assemble a complete supervised-learning mini-project: EDA → preprocessing → model → evaluation.

3. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Supervised learning concepts; the Scikit-learn API; train/test split
Day 2	8 hrs	Linear regression: fitting, predicting, coefficients, regression metrics
Day 3	8 hrs	Logistic regression & classification; confusion matrix and classification metrics
Day 4	8 hrs	Decision trees, random forests, SVMs, k-NN; comparing classifiers
Day 5	8 hrs	Supervised-learning mini-project: full pipeline on a real dataset

4. Day-by-Day Curriculum

Day 1 — Supervised Learning Concepts & the Scikit-learn API — 8 hours

Learning Objectives

- Explain supervised learning and distinguish regression from classification.
- Separate a dataset into features (X) and target (y).
- Perform a train/test split and explain why evaluating on unseen data is essential.

Key Topics

- What supervised learning is: learning from labeled examples
- Regression vs. classification
- Features (X) and target (y)
- The consistent Scikit-learn API: instantiate, fit, predict, score
- The train/test split and why it matters

Lesson Content

1.1 What Supervised Learning Is

Supervised learning trains a model on labeled examples — data where the correct answer (the target) is already known — so it can predict that answer for new, unseen data. "Supervised" means every training example comes with its correct label, like a teacher's answer key. This is the most common and immediately useful category of ML.

1.2 Regression vs. Classification

The two types of supervised learning are distinguished entirely by what the target looks like:

	Regression	Classification
Predicts	A continuous number	A category / class
Example	House price, temperature, income	Spam / not spam, churn / stay, disease type
Example metric	RMSE, MAE, R^2	Accuracy, precision, recall, F1
This week	Day 2	Days 3-4

1.3 Features and Target

The inputs are the features (X) — the columns the model learns from. The output being predicted is the target (y) — the single column of correct answers. The entire task of supervised learning is learning the relationship between X and y from training data.

```
X = df.drop("target", axis=1) # features: everything except the target
y = df["target"]             # target: the column to predict
```

1.4 The Scikit-learn API

Scikit-learn is the standard classical-ML library, and its greatest strength is a consistent API: every model — no matter how different internally — is used through the same four steps. Learn it once, and you can use every model in the library.

Step	Method	What It Does
1. Instantiate	<code>model = Model()</code>	Create the model, set its options
2. Fit	<code>model.fit(X_train, y_train)</code>	Learn patterns from training data
3. Predict	<code>model.predict(X_test)</code>	Predict on new data
4. Score	<code>model.score(X_test, y_test)</code>	Evaluate performance

1.5 The Train/Test Split

The single most important rule in ML: never evaluate a model on the same data it was trained on. A model can memorize its training data and look perfect, yet fail completely on new data. Splitting the data — training on one part, evaluating on a held-out part it has never seen — is the only honest way to estimate real-world performance.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

`test_size=0.2` holds out 20% for testing; `random_state=42` fixes the split so results are reproducible — the same reproducibility discipline established in Week 1.

Hands-On Lab: Setting Up the ML Workflow

- Step 1: Load a provided dataset and separate it into features `X` and target `y`.
- Step 2: Perform an 80/20 train/test split with a fixed `random_state`.
- Step 3: Confirm the shapes of `X_train`, `X_test`, `y_train`, `y_test` are consistent and print them.
- Step 4: In a Markdown cell, explain in your own words why the model must never see the test set during training.

Tools Used

Scikit-learn • Pandas • Jupyter Notebook

Day 2 — Linear Regression — 8 hours

Learning Objectives

- Train a linear regression model and generate predictions.
- Interpret the model's coefficients and intercept.
- Evaluate a regression model with MAE, RMSE, and R^2 against a baseline.

Key Topics

- What linear regression does: fitting the best line
- Training and predicting with Scikit-learn
- Interpreting coefficients and the intercept
- Regression metrics: MAE, RMSE, R^2
- Comparing against a baseline

Lesson Content

2.1 What Linear Regression Does

Linear regression predicts a continuous number by fitting the best straight line (or hyperplane) through the data. Its prediction is exactly the dot product from Week 2: $\text{prediction} = (\text{feature}_1 \times \text{weight}_1) + (\text{feature}_2 \times \text{weight}_2) + \dots + \text{bias}$. Training means finding the weights that make the line fit the data as closely as possible.

2.2 Training and Predicting

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

2.3 Interpreting Coefficients

After fitting, each feature has a coefficient (its weight) and the model has one intercept (the bias). A coefficient tells you how much the prediction changes when that feature increases by one unit, holding others constant — this interpretability is a major reason linear regression remains widely used even in the deep-learning era.

```
print(model.coef_)      # one weight per feature
print(model.intercept_) # the bias term
```

2.4 Regression Metrics

A regression model's error is measured by how far its predictions land from the true values. The three standard metrics:

Metric	Meaning	Interpretation
MAE	Mean Absolute Error	Average size of the error, in the target's units — easy to explain
RMSE	Root Mean Squared Error	Like MAE but penalizes large errors more heavily
R ²	Coefficient of determination	Fraction of variance explained; 1.0 is perfect, 0 is no better than the mean

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
print(mean_absolute_error(y_test, predictions))
print(np.sqrt(mean_squared_error(y_test, predictions))) # RMSE
print(r2_score(y_test, predictions))
```

Always compare a model's metric against a simple baseline (e.g. predicting the mean for every row). A model that cannot beat the baseline has learned nothing useful.

Hands-On Lab: Predicting a Continuous Value

- Step 1: Train a LinearRegression model on a provided regression dataset (e.g. house prices).
- Step 2: Report the model's coefficients and identify which feature has the strongest effect.
- Step 3: Evaluate the model with MAE, RMSE, and R² on the test set.
- Step 4: Compare the RMSE against a baseline that predicts the mean for every row, and state whether the model adds value.
- Step 5: Document the interpretation of results in Markdown.

Tools Used

Scikit-learn (LinearRegression) • Pandas • Jupyter Notebook

Day 3 — Logistic Regression & Classification Metrics — 8 hours

Learning Objectives

- Train a logistic regression classifier and obtain class probabilities.
- Explain why accuracy alone is misleading on imbalanced data.
- Read a confusion matrix and compute precision, recall, F1, and AUC-ROC.

Key Topics

- Logistic regression: weighted sum + sigmoid → probability
- Why accuracy is misleading on imbalanced data
- The confusion matrix: TP, FP, FN, TN
- Precision, recall, F1, and their trade-off
- AUC-ROC

Lesson Content

3.1 From Regression to Classification

Despite its name, logistic regression is a classification model — it predicts a category, not a number. It computes the same weighted sum as linear regression, then passes it through the sigmoid function, which squashes any number into a probability between 0 and 1. If that probability exceeds a threshold (usually 0.5), it predicts the positive class.

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
probabilities = model.predict_proba(X_test) # class probabilities
```

3.2 Why Accuracy Alone Is Misleading

Accuracy — the fraction of correct predictions — is the obvious metric but is dangerously misleading on imbalanced data. If 95% of customers don't churn, a model that always predicts "no churn" scores 95% accuracy while being completely useless. This is why classification needs richer metrics.

3.3 The Confusion Matrix

The confusion matrix is the foundation of all classification metrics — it breaks predictions into four categories:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN) — missed
Actual Negative	False Positive (FP) — false alarm	True Negative (TN)

3.4 Precision, Recall & F1

Metric	Formula	Answers the Question
Precision	$TP / (TP + FP)$	Of everything I predicted positive, how much was right?
Recall	$TP / (TP + FN)$	Of all actual positives, how many did I catch?
F1-score	harmonic mean of the two	A single balanced score when you need both

The trade-off matters and depends on the problem: for disease screening, recall matters most (never miss a real case); for a spam filter, precision matters most (never block a real email). Choosing which to optimize is a real decision, not a formula.

```
from sklearn.metrics import confusion_matrix, classification_report
print(confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions))
```

3.5 AUC-ROC

The ROC curve plots the trade-off between catching positives and raising false alarms across every threshold; the AUC (Area Under the Curve) summarizes it as a single number from 0.5 (random guessing) to 1.0 (perfect). It is the standard metric for comparing classifiers independently of any single threshold choice.

Hands-On Lab: Building a Classifier

- Step 1: Train a LogisticRegression model on a provided classification dataset (e.g. churn).
- Step 2: Generate predictions and produce the confusion matrix.
- Step 3: Compute precision, recall, and F1 with classification_report, and interpret each.
- Step 4: Decide whether precision or recall matters more for this specific problem and justify it.
- Step 5: Compute the AUC-ROC and document what it says about the model.

Tools Used

Scikit-learn (LogisticRegression) • Pandas • Matplotlib • Jupyter Notebook

Day 4 — Trees, Forests, SVMs & k-NN — 8 hours

Learning Objectives

- Train and interpret decision trees and random forests, including feature importances.
- Train SVM and k-NN classifiers and explain how each makes decisions.
- Compare multiple classifiers fairly on the same train/test split and metric.

Key Topics

- Decision trees: rule-based, interpretable, prone to overfitting
- Random forests: ensembles and feature importances
- Support Vector Machines and the margin
- k-Nearest Neighbors
- Comparing models fairly (no free lunch)

Lesson Content

4.1 Decision Trees

A decision tree predicts by asking a sequence of yes/no questions about the features, splitting the data at each step until it reaches a decision. Its great strength is interpretability — you can read the exact rules it learned. Its weakness is a tendency to overfit: a deep tree can memorize the training data and generalize poorly.

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=5, random_state=42)
model.fit(X_train, y_train)
```

4.2 Random Forests

A random forest is an ensemble — it trains many decision trees on random subsets of the data and features, then averages their votes. This "wisdom of the crowd" fixes the single tree's overfitting problem and is one of the strongest, most reliable classical models available, often a great default choice.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print(model.feature_importances_) # which features mattered most
```

Random forests also report feature importances, showing which inputs drove predictions most — valuable both for

insight and for explaining the model to non-technical stakeholders.

4.3 Support Vector Machines (SVM)

An SVM finds the decision boundary that maximizes the margin — the widest possible gap between the two classes. With the "kernel trick" it can also draw curved boundaries for data that isn't linearly separable. SVMs are powerful on small-to-medium, high-dimensional datasets but scale poorly to very large ones.

```
from sklearn.svm import SVC
model = SVC(kernel="rbf", probability=True)
model.fit(X_train, y_train)
```

4.4 k-Nearest Neighbors (k-NN)

k-NN is the simplest classifier: to predict a new point's class, it looks at the k closest training points and takes a majority vote. It learns nothing during training — all the work happens at prediction time. It is intuitive and effective on small datasets but slows down and degrades as data grows in size or dimensions.

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```

4.5 Comparing Models

No single algorithm is best for every problem (the "no free lunch" principle). The professional approach is to train several models on the same train/test split and compare them fairly on the same metric — exactly what the Day 4 lab does.

Model	Strength	Watch Out For
Decision Tree	Interpretable rules	Overfitting if too deep
Random Forest	Strong, reliable default	Less interpretable than one tree
SVM	Powerful in high dimensions	Slow on large datasets
k-NN	Simple, intuitive	Slow and weaker as data grows

Hands-On Lab: Model Comparison

- Step 1: Train a decision tree, random forest, SVM, and k-NN on the same train/test split from Day 3.
- Step 2: Evaluate all four with the same metric (e.g. F1-score) and assemble the results into one comparison table.
- Step 3: Report the random forest's top feature importances and interpret them.
- Step 4: Identify the best-performing model for this dataset and explain, in Markdown, why it likely won.

Tools Used

Scikit-learn (tree, ensemble, svm, neighbors) • Pandas • Jupyter Notebook

Day 5 — Supervised-Learning Mini-Project — 8 hours

Learning Objectives

- Assemble a complete supervised-learning pipeline from EDA to evaluation.
- Apply basic preprocessing (encoding, scaling) without data leakage.
- Select and justify an appropriate model and metric for the task, and document the result.

Key Topics

- The full pipeline: EDA → preprocessing → split → model → evaluation
- Basic preprocessing: one-hot encoding, feature scaling
- Avoiding data leakage (fit scaler on train only)
- Choosing the right model and metric for the task
- Documenting the result against a baseline

Lesson Content

5.1 The Full Supervised-Learning Pipeline

Week 3 closes by combining everything into one end-to-end mini-project — the same pipeline shape every project in this program follows, and the exact structure the Phase 3 capstone will formalize:

Stage	What Happens
1. EDA	Understand the data (Week 2 methodology): distributions, correlations, problems
2. Preprocessing	Handle missing values, encode categories, scale features
3. Train/test split	Hold out unseen data for honest evaluation
4. Modeling	Train one or more models via the Scikit-learn API
5. Evaluation	Measure performance with appropriate metrics vs. a baseline

5.2 Basic Preprocessing

Two preprocessing steps appear in almost every project. Categorical features (text categories) must be converted to numbers, since models only accept numeric input; one-hot encoding is the standard method. Feature scaling puts all numeric features on a comparable range, which matters greatly for distance- and margin-based models like k-NN and SVM.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # note: transform only, never fit on test
```

Critical rule: fit the scaler on the training data only, then apply it to the test data. Fitting on the full dataset leaks information from the test set into training and inflates results — a subtle but serious mistake.

5.3 Choosing the Right Model and Metric

The mini-project requires two real decisions: whether the task is regression or classification (determining which models and metrics apply), and which metric best reflects success for this specific problem. Making and justifying these choices — not just running code — is what the mentor evaluates.

5.4 Documenting the Result

The deliverable is a clean, narrated notebook: each stage explained in Markdown, each choice justified, and the final result compared honestly against a baseline. This is the same standard applied to every notebook in the program and the foundation for the technical write-ups in Phase 3.

Hands-On Lab: End-to-End Mini-Project

- Step 1: Choose a provided dataset and determine whether the task is regression or classification.
- Step 2: Perform brief EDA, then preprocess: handle missing values, encode categoricals, and scale features (fit on train only).
- Step 3: Train at least two appropriate models and evaluate them with a suitable metric against a baseline.



- Step 4: Select the better model, justify the choice, and document the full pipeline in a narrated notebook.
- Step 5: Commit the finished mini-project notebook to GitHub with a clear commit message.

Tools Used

Scikit-learn • Pandas • Matplotlib / Seaborn • Jupyter Notebook • Git & GitHub

BinX
TECH

5. Week 3 Deliverables

By the end of Week 3, every intern must submit the following to their mentor and GitHub repository:

- A workflow notebook demonstrating the features/target split and a reproducible train/test split.
- A linear regression notebook with interpreted coefficients and MAE/RMSE/R² evaluated against a baseline.
- A logistic regression notebook with a confusion matrix, precision/recall/F1, and AUC-ROC.
- A model-comparison notebook evaluating decision tree, random forest, SVM, and k-NN on one split.
- The Week 3 end-to-end mini-project: a trained, evaluated model with full pipeline documentation.
- All Week 3 notebooks committed to the intern's GitHub repository with clear commit messages.

6. Week 3 Evaluation Criteria

Scored by the assigned mentor at the end of Week 3 using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Understanding of ML concepts	Runs models but unclear on how they work	Explains model choices and metrics clearly	Deep understanding; justifies model and metric trade-offs
Scikit-learn proficiency	Uses the API with guidance	Trains and evaluates models independently	Fluent; builds clean pipelines without data leakage
Model evaluation & metric reporting	Single metric, no baseline	Multiple metrics with a baseline comparison	Full evaluation with the right metric for the problem
Preprocessing correctness	Raw or leaky preprocessing	Correct encoding and scaling, fit on train only	Thorough, justified, leakage-free preprocessing
Notebook quality & Git workflow	Sporadic commits, weak narrative	Regular commits, clear Markdown narrative	Consistent, descriptive, reproducible, well-organized
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

TECH

7. Technical Stack — Week 3

Core ML Library	Scikit-learn (LinearRegression, LogisticRegression, tree, ensemble, svm, neighbors)
Preprocessing	StandardScaler, one-hot encoding, train_test_split
Metrics	MAE, RMSE, R^2 , confusion_matrix, classification_report, AUC-ROC
Data & Viz	Pandas, Matplotlib, Seaborn
Environment	Python 3.10+, Jupyter Notebook, Git & GitHub

8. Notes & Best Practices

This is the week interns train their first real models, so two habits matter most: always evaluate on a held-out test set, and always compare against a baseline. A model is only meaningful relative to what it beats. The Day 5 mini-project is the first true model deliverable and a direct rehearsal for the Phase 3 capstone pipeline.